

Bagaimana *Physics-Informed Neural Networks* (PINNs) bekerja?

Fajar Adie Santosa (@debug.stack)

August 2, 2026

Contents

1	Pengertian PINNs	4
2	Konsep Dasar PINNs	4
2.1	Arsitektur Neural Network	4
2.2	Hard Constraint	4
2.3	Soft Constraint	5
2.4	Fungsi Loss	6
2.5	Tahapan Kerja PINNs	6
3	Problem 1: Persamaan Poisson 1D	7
3.1	Problem Setup	7
3.2	Syarat Batas	7
3.3	Solusi Analitik	7
3.4	Loss Residual	7
3.5	Arsitektur dan Optimasi	7
3.6	Penjelasan Kode Langkah demi Langkah	8
3.6.1	Membangun jaringan dan menerapkan syarat batas . .	8
3.6.2	Menghitung residual dengan automatic differentiation .	8
3.6.3	Menyiapkan titik kolokasi dan melakukan optimasi . .	9
3.6.4	Evaluasi solusi	9
4	Problem 2: Persamaan Bratu 1D	10
4.1	Problem Setup	10
4.2	Syarat Batas	10
4.3	Solusi Analitik	10
4.4	Loss Residual	10
4.5	Arsitektur dan Optimasi	11
4.6	Penjelasan Kode Langkah demi Langkah	12
4.6.1	Membangun model dengan hard constraint	12
4.6.2	Membentuk turunan dan residual Bratu	12
4.6.3	Menentukan titik kolokasi dan melatih model	13
4.6.4	Mengevaluasi hasil	13
5	Problem 3: Osilator Harmonik Teredam	14
5.1	Dasar Fisika	14
5.2	Syarat Awal	15

5.3	Solusi Analitik	15
5.4	Formulasi PINN pada Notebook	15
5.5	Penjelasan Kode Langkah demi Langkah	16
5.5.1	Membangkitkan solusi analitik	16
5.5.2	Membangun neural network	17
5.5.3	Mendefinisikan data loss dan physics loss	17
5.5.4	Menyiapkan data dan titik kolokasi	18
5.5.5	Optimasi tahap pertama dengan Adam	18
5.5.6	Penyempurnaan dengan L-BFGS	19
5.5.7	Mengukur galat solusi	19

1 Pengertian PINNs

Physics-Informed Neural Networks (PINNs) merupakan gabungan antara *neural networks* dengan hukum fisika yang dideskripsikan melalui persamaan diferensial. Hukum fisika tersebut dimasukkan ke dalam fungsi *loss* selama proses pelatihan, sehingga solusi yang dihasilkan tetap konsisten dengan persamaan diferensial yang berlaku [1]. PINNs dapat digunakan untuk:

- Pendekatan solusi untuk persamaan diferensial biasa (PDB) dan persamaan diferensial parsial (PDP).
- Menyelesaikan *inverse problems*, seperti estimasi parameter model dari data yang terbatas.

2 Konsep Dasar PINNs

2.1 Arsitektur Neural Network

Andaikan $u_{nn}(x; \theta)$ merupakan keluaran dari *neural network* dengan parameter θ . Secara umum, PINNs menggunakan arsitektur *fully connected network* (FCN) dengan fungsi aktivasi tanh:

$$u_{nn}(x; \theta) = W_L \sigma(W_{L-1} \cdots \sigma(W_1 x + b_1) \cdots + b_{L-1}) + b_L \quad (1)$$

di mana W_i dan b_i masing-masing adalah bobot dan bias pada lapisan ke- i , dan σ adalah fungsi aktivasi.

2.2 Hard Constraint

Untuk memastikan solusi memenuhi syarat batas atau syarat awal secara otomatis, digunakan pendekatan *hard constraint*. Solusi dibentuk sebagai:

$$u_{PINN}(x; \theta) = p(x) \cdot u_{nn}(x; \theta) \quad (2)$$

di mana $p(x)$ dipilih sedemikian sehingga u_{PINN} secara aljabar memenuhi syarat batas. Dengan pendekatan ini, *loss* dari syarat batas tidak perlu dimasukkan ke dalam fungsi *loss* secara eksplisit.

2.3 Soft Constraint

Alternatif lain adalah menggunakan pendekatan *soft constraint*, di mana syarat batas dimasukkan ke dalam fungsi *loss* sebagai penalti:

$$\mathcal{L} = \mathcal{L}_{PDE} + \lambda_{BC}\mathcal{L}_{BC} \quad (3)$$

dengan λ_{BC} sebagai bobot penalti untuk syarat batas. Berbeda dengan *hard constraint*, keluaran jaringan tidak dipaksa memenuhi syarat batas secara aljabar. Jaringan tetap menghasilkan aproksimasi langsung $u_{nn}(x; \theta)$, kemudian pelatihan diarahkan agar residual PDE dan residual syarat batas sama-sama kecil.

Misalnya untuk masalah satu dimensi dengan syarat batas Dirichlet

$$u(a) = u_a, \quad u(b) = u_b, \quad (4)$$

komponen *boundary loss* dapat ditulis sebagai:

$$\mathcal{L}_{BC} = |u_{nn}(a; \theta) - u_a|^2 + |u_{nn}(b; \theta) - u_b|^2. \quad (5)$$

Apabila syarat yang digunakan adalah syarat awal, bentuknya serupa, hanya saja titik evaluasinya berada pada waktu awal. Sebagai contoh, untuk $u(0) = u_0$ dan $u'(0) = v_0$, penalti syarat awal dapat ditulis:

$$\mathcal{L}_{IC} = |u_{nn}(0; \theta) - u_0|^2 + \left| \frac{du_{nn}}{dt}(0; \theta) - v_0 \right|^2. \quad (6)$$

Dengan demikian, bentuk umum *loss* pada PINNs dengan *soft constraint* adalah:

$$\mathcal{L} = \lambda_{PDE}\mathcal{L}_{PDE} + \lambda_{BC}\mathcal{L}_{BC} + \lambda_{IC}\mathcal{L}_{IC}, \quad (7)$$

di mana setiap λ mengatur seberapa besar prioritas masing-masing komponen selama optimasi. Nilai λ yang terlalu kecil dapat membuat syarat batas atau syarat awal tidak terpenuhi dengan baik, sedangkan nilai yang terlalu besar dapat membuat pelatihan terlalu fokus pada syarat batas dan mengabaikan residual PDE di domain.

Keuntungan utama *soft constraint* adalah bentuknya lebih fleksibel karena tidak perlu merancang fungsi pembentuk seperti $p(x)$ pada *hard constraint*. Namun, syarat batas hanya dipenuhi secara pendekatan, bukan secara eksak. Oleh karena itu, setelah pelatihan selesai, nilai u_{nn} pada titik batas tetap perlu diperiksa untuk memastikan galat syarat batas cukup kecil.

2.4 Fungsi Loss

Fungsi *loss* PINNs terdiri atas residual persamaan diferensial yang dievaluasi di titik-titik kolokasi $\{x_i\}_{i=1}^N$:

$$\mathcal{L} = \mathcal{L}_{PDE} = \frac{1}{N} \sum_{i=1}^N |\mathcal{F}[u_{PINN}](x_i)|^2 \quad (8)$$

di mana $\mathcal{F}[\cdot]$ adalah operator diferensial dari persamaan yang diselesaikan. Apabila tidak menggunakan *hard constraint*, fungsi *loss* juga mencakup residual syarat batas:

$$\mathcal{L} = \mathcal{L}_{PDE} + \lambda \mathcal{L}_{BC} \quad (9)$$

2.5 Tahapan Kerja PINNs

Secara umum, proses penyelesaian persamaan diferensial dengan PINNs terdiri atas tahapan berikut:

1. Menentukan persamaan diferensial, domain, serta syarat batas atau syarat awal dari masalah fisika.
2. Membentuk jaringan saraf $u_{nn}(\mathbf{x}; \theta)$ sebagai aproksimasi solusi. Jika digunakan *hard constraint*, keluaran jaringan terlebih dahulu dikalikan atau dijumlahkan dengan fungsi pembatas yang sesuai.
3. Memilih titik kolokasi di dalam domain. Titik ini tidak harus memiliki label solusi karena digunakan untuk mengevaluasi residual persamaan diferensial.
4. Menghitung turunan keluaran jaringan terhadap masukan menggunakan *automatic differentiation*, kemudian menyusun residual $\mathcal{R}$.
5. Meminimalkan rata-rata kuadrat residual, serta komponen *loss* lain jika digunakan, terhadap parameter θ .
6. Mengevaluasi solusi pada titik uji dan membandingkannya dengan solusi analitik menggunakan metrik galat.

Perbedaan utama PINNs dan jaringan saraf biasa terletak pada informasi yang digunakan selama pelatihan. Jaringan saraf biasa terutama memerlukan pasangan masukan–target (x_i, y_i) , sedangkan PINNs juga dapat belajar dari titik kolokasi tanpa target melalui residual hukum fisika. Dengan

demikian, gradien parameter jaringan tidak hanya membawa informasi kecocokan terhadap data, tetapi juga ketidakpatuhan solusi terhadap persamaan diferensial.

3 Problem 1: Persamaan Poisson 1D

3.1 Problem Setup

PDE:

$$\frac{\partial^2 y}{\partial x^2} + \pi^2 \sin(\pi x) = 0, \quad x \in [-1, 1] \quad (10)$$

3.2 Syarat Batas

$$y(-1) = 0, \quad y(1) = 0 \quad (11)$$

3.3 Solusi Analitik

$$y(x) = \sin(\pi x) \quad (12)$$

3.4 Loss Residual

Andaikan y_{nn} merupakan keluaran *neural network*. Residual persamaan diferensial didefinisikan sebagai:

$$\mathcal{R}(x) = \frac{\partial^2 y_{nn}}{\partial x^2} + \pi^2 \sin(\pi x) \approx 0 \quad (13)$$

Karena menggunakan *hard constraint* dengan $p(x) = 1 - x^2$, syarat batas $y(\pm 1) = 0$ dipenuhi secara otomatis. Sehingga fungsi *loss* hanya terdiri atas residual PDE:

$$\mathcal{L} = \mathcal{L}_{PDE} = \frac{1}{N} \sum_{i=1}^N \mathcal{R}(x_i)^2 \quad (14)$$

3.5 Arsitektur dan Optimasi

Model yang digunakan adalah FCN dengan arsitektur $[1, 10, 10, 1]$ dan fungsi aktivasi tanh. Optimasi dilakukan menggunakan algoritma L-BFGS dengan *automatic differentiation* untuk menghitung turunan kedua $\frac{\partial^2 y_{nn}}{\partial x^2}$ melalui `tf.GradientTape`.

3.6 Penjelasan Kode Langkah demi Langkah

3.6.1 Membangun jaringan dan menerapkan syarat batas

Fungsi `build_fcn` menerima daftar ukuran lapisan. Dua lapisan tersembunyi menggunakan aktivasi `tanh`, sedangkan lapisan keluaran bersifat linear. Keluaran mentah jaringan kemudian dikalikan dengan $1 - x^2$.

```
1 inputs = tf.keras.Input(shape=(layer_sizes[0],))
2 a = inputs
3
4 for units in layer_sizes[1:-1]:
5     a = layers.Dense(units, activation='tanh',
6                       kernel_initializer=initializers.GlorotNormal(),
7                       bias_initializer='zeros')(a)
8
9 nn_out = layers.Dense(layer_sizes[-1], activation=None)(a)
10 constraint = layers.Lambda(lambda x: 1.0 - tf.square(x))(
11     inputs)
12 outputs = layers.Multiply()([constraint, nn_out])
13 model = Model(inputs=inputs, outputs=outputs)
```

Listing 1: FCN dan hard constraint pada masalah Poisson

Pada $x = -1$ maupun $x = 1$, faktor $1 - x^2$ bernilai nol. Oleh sebab itu, `model(x)` selalu memenuhi $y(-1) = y(1) = 0$ untuk nilai parameter jaringan apa pun.

Catatan implementasi: cell asli juga memuat baris `model = models.Sequential()`, tetapi modul `models` tidak diimpor dan objek tersebut tidak dipakai. Jika notebook dijalankan ulang dari awal, baris itu perlu dihapus atau impor yang sesuai perlu ditambahkan. Uraian di atas mengikuti Functional API yang benar-benar dikembalikan oleh fungsi, yaitu `Model(inputs=inputs, outputs=outputs)`. Berkas notebook tidak diubah dalam penyusunan dokumen ini.

3.6.2 Menghitung residual dengan automatic differentiation

Dua `GradientTape` yang bersarang menghitung turunan pertama dan kedua. Opsi `watch(x)` diperlukan karena turunan yang dicari adalah terhadap masukan x , bukan terhadap parameter model.

```
1 def PDE(x, model):
2     with tf.GradientTape(persistent=True) as tape2:
3         tape2.watch(x)
```



```

4         with tf.GradientTape(persistent=True) as tape1:
5             tape1.watch(x)
6             y_pred = model(x)
7             y_x = tape1.gradient(y_pred, x)
8             y_xx = tape2.gradient(y_x, x)
9             return y_xx + (np.pi**2) * tf.sin(np.pi * x)
10
11 def lossPDE(model, xPDE):
12     f = PDE(xPDE, model)
13     return tf.reduce_mean(tf.square(f))

```

Listing 2: Residual Poisson dan loss PDE

Nilai yang dikembalikan oleh PDE adalah $\mathcal{R}(x)$. Pelatihan mengecilkan $\text{mean}(\text{square}(f))$, yaitu aproksimasi diskret dari $\mathcal{L}_{PDE}$ pada titik kolokasi.

3.6.3 Menyiapkan titik kolokasi dan melakukan optimasi

Kode membentuk 100 titik berjarak seragam pada $[-1, 1]$, kemudian membangun arsitektur $[1, 10, 10, 1]$. Kelas `LBFGSOptimizer` meratakan seluruh bobot dan bias menjadi satu vektor karena `tfp.optimizer.lbfgs_minimize` menerima parameter dalam bentuk tersebut.

```

1 x_train = np.linspace(-1, 1, 100).reshape(-1, 1)
2 x_train = tf.convert_to_tensor(x_train, dtype=tf.float32)
3
4 model = build_fcn([1, 10, 10, 1])
5
6 def loss_fn(model):
7     return loss(model, x_train, x_bc, y_bc)
8
9 optimizer = LBFGSOptimizer(model, loss_fn)
10 optimizer.minimize(max_iterations=1000)

```

Listing 3: Titik kolokasi dan pelatihan Poisson

Argumen `x_bc` dan `y_bc` tetap diteruskan ke fungsi `loss`, tetapi tidak digunakan karena syarat batas sudah diterapkan secara eksak oleh *hard constraint*.

3.6.4 Evaluasi solusi

Prediksi dihitung pada 1000 titik uji dan dibandingkan dengan $\sin(\pi x)$. Galat relatif L^2 menormalkan besar selisih dengan norma solusi analitik, se-

hingga nilainya dapat dibandingkan tanpa bergantung langsung pada skala solusi.

```

1 y_pred = model(x_test).numpy()
2 y_analytic = np.sin(np.pi * x_test)
3 l2_error = (np.linalg.norm(y_pred - y_analytic)
4           / np.linalg.norm(y_analytic))

```

Listing 4: Evaluasi solusi Poisson

4 Problem 2: Persamaan Bratu 1D

4.1 Problem Setup

PDE:

$$\frac{d^2 u}{dx^2} + Ce^u = 0, \quad x \in (0, 1) \quad (15)$$

dengan ukuran titik kolokasi $h = \frac{1}{n}$.

4.2 Syarat Batas

$$u(0) = 0, \quad u(1) = 0 \quad (16)$$

4.3 Solusi Analitik

Berdasarkan referensi Shahab et al. (2025), untuk nilai $C = 1$:

$$u(x) = 2 \ln \left(\frac{\cosh(\theta)}{\cosh(\theta(1 - 2x))} \right) \quad (17)$$

di mana nilai θ diperoleh dari:

$$\frac{4\theta}{\sqrt{2C}} = \cosh(\theta), \quad \theta \approx 0.379291 \quad (18)$$

4.4 Loss Residual

Andaikan $\hat{u}(x; \theta)$ merupakan aproksimasi solusi yang dihasilkan oleh PINNs. Residual persamaan Bratu didefinisikan sebagai:

$$\mathcal{R}(x) = \frac{d^2 \hat{u}}{dx^2}(x; \theta) + Ce^{\hat{u}(x; \theta)} \approx 0. \quad (19)$$

Untuk masalah Bratu 1D, formulasi *hard constraint* dan *soft constraint* dapat dibandingkan sebagai berikut:

Hard Constraint	Soft Constraint
Solusi dibentuk dengan fungsi pembatas: $\hat{u}(x; \theta) = x(1 - x)u_{nn}(x; \theta).$ Karena $x(1 - x) = 0$ pada $x = 0$ dan $x = 1$, maka syarat batas $u(0) = u(1) = 0$ terpenuhi secara otomatis.	Solusi langsung diambil dari keluaran jaringan: $\hat{u}(x; \theta) = u_{nn}(x; \theta).$ Syarat batas tidak dipaksa secara aljabar, tetapi dimasukkan sebagai penalti dalam fungsi <i>loss</i>.
Fungsi <i>loss</i> hanya memuat residual PDE: $\mathcal{L} = \mathcal{L}_{PDE} = \frac{1}{N} \sum_{i=1}^N \mathcal{R}(x_i)^2.$	Fungsi <i>loss</i> memuat residual PDE dan residual syarat batas: $\mathcal{L} = \mathcal{L}_{PDE} + \lambda_{BC} \mathcal{L}_{BC},$ dengan $\mathcal{L}_{BC} = u_{nn}(0; \theta) ^2 + u_{nn}(1; \theta) ^2.$
Syarat batas dipenuhi secara eksak, sehingga tidak perlu memilih bobot <i>loss</i> untuk syarat batas.	Syarat batas dipenuhi secara pendekatan. Nilai λ_{BC} perlu dipilih dengan hati-hati agar model tidak terlalu mengabaikan PDE maupun syarat batas.

4.5 Arsitektur dan Optimasi

Model yang digunakan adalah FCN dengan arsitektur $[1, 20, 20, 1]$ dan fungsi aktivasi tanh. Optimasi dilakukan menggunakan L-BFGS (`torch.optim.LBFGS`) dengan parameter `history_size = 50` dan `line_search_fn = "strong_wolfe"`. Turunan dihitung menggunakan *automatic differentiation* dari PyTorch.

4.6 Penjelasan Kode Langkah demi Langkah

4.6.1 Membangun model dengan hard constraint

Kelas PINN membentuk jaringan $[1, 20, 20, 1]$. Metode `forward` mengalikan keluaran mentah jaringan dengan $x(1-x)$ sehingga kedua syarat batas selalu terpenuhi.

```
1 self.net = nn.Sequential(  
2     nn.Linear(1, n_hidden), nn.Tanh(),  
3     nn.Linear(n_hidden, n_hidden), nn.Tanh(),  
4     nn.Linear(n_hidden, 1)  
5 )  
6  
7 def forward(self, x):  
8     nn_out = self.net(x)  
9     constraint = x * (1 - x)  
10    return constraint * nn_out
```

Listing 5: Arsitektur PINN untuk persamaan Bratu

Bobot dan bias lapisan terakhir dikalikan 0.01 sebelum pelatihan. Inisialisasi ini membuat keluaran awal mendekati nol dan membantu menjaga nilai $\exp(u)$ agar tidak terlalu besar pada awal optimasi.

4.6.2 Membentuk turunan dan residual Bratu

Fungsi `derivative` menggunakan mekanisme *autograd* PyTorch. Opsi `create_graph` diaktifkan agar grafik komputasi turunan tetap disimpan. Dengan demikian, u_x dapat diturunkan kembali untuk memperoleh u_{xx} .

```
1 def derivative(y, x):  
2     return torch.autograd.grad(  
3         y, x, grad_outputs=torch.ones_like(y),  
4         retain_graph=True, create_graph=True)[0]  
5  
6 def physics_loss(model, x, C):  
7     u = model(x)  
8     u_x = derivative(u, x)  
9     u_xx = derivative(u_x, x)  
10    f = u_xx + C * torch.exp(u)  
11    return torch.mean(f**2)
```

Listing 6: Automatic differentiation dan residual Bratu

4.6.3 Menentukan titik kolokasi dan melatih model

Sebanyak 200 titik interior dipilih dari h sampai $1 - h$, dengan $h = 1/200$. Atribut `requires_grad_(True)` wajib diaktifkan agar PyTorch dapat menghitung turunan solusi terhadap koordinat kolokasi.

```
1 n_col = 200
2 h_col = 1 / n_col
3 x_train = np.linspace(h_col, 1 - h_col, n_col).reshape(-1, 1)
4 x_train = torch.tensor(x_train, dtype=torch.float32, device=
    device)
5 x_train.requires_grad_(True)
6
7 optimizer = torch.optim.LBFGS(
8     model.parameters(), max_iter=2000, max_eval=1250,
9     history_size=50, tolerance_grad=1e-10,
10    tolerance_change=1e-12, line_search_fn="strong_wolfe"
11 )
```

Listing 7: Titik kolokasi dan optimizer L-BFGS untuk Bratu

L-BFGS memerlukan fungsi *closure* karena optimizer dapat mengevaluasi nilai fungsi dan gradien beberapa kali dalam satu pemanggilan `optimizer.step`. Setiap evaluasi menghapus gradien lama, menghitung *physics loss*, menjalankan *backpropagation*, lalu mengembalikan nilai *loss*.

```
1 def closure():
2     optimizer.zero_grad()
3     loss = physics_loss(model, x_train, C=C)
4     loss.backward()
5     loss_log.append(loss.item())
6     return loss
7
8 loss = optimizer.step(closure)
```

Listing 8: Closure pelatihan Bratu

4.6.4 Mengevaluasi hasil

Model dievaluasi pada 200 titik di $[0, 1]$. Karena evaluasi tidak memerlukan gradien, blok `torch.no_grad()` mengurangi penggunaan memori. Prediksi kemudian dibandingkan dengan solusi analitik menggunakan galat relatif L^2 .

```
1 with torch.no_grad():
2     u_pred = model(x_plot_tensor).numpy()
3
```

```

4 u_analytic = 2 * np.log(
5     np.cosh(theta) / np.cosh(theta * (1 - 2 * x_plot)))
6 l2_error = (np.linalg.norm(u_pred - u_analytic)
7     / np.linalg.norm(u_analytic))

```

Listing 9: Evaluasi solusi Bratu

5 Problem 3: Osilator Harmonik Teredam

5.1 Dasar Fisika

Osilator harmonik teredam dapat dimodelkan sebagai sebuah massa yang terhubung dengan pegas dan peredam. Berdasarkan hukum II Newton, resultan gaya pegas $-ku$ dan gaya redaman $-\mu\dot{u}$ sama dengan $m\ddot{u}$, sehingga:

$$m \frac{d^2 u}{dt^2} + \mu \frac{du}{dt} + ku = 0, \quad (20)$$

di mana m adalah massa, μ adalah koefisien redaman, dan k adalah konstanta pegas. Dengan mendefinisikan

$$\delta = \frac{\mu}{2m}, \quad \omega_0 = \sqrt{\frac{k}{m}}, \quad (21)$$

persamaan tersebut dapat ditulis sebagai

$$\frac{d^2 u}{dt^2} + 2\delta \frac{du}{dt} + \omega_0^2 u = 0. \quad (22)$$

Energi mekanik sistem adalah

$$E(t) = \frac{1}{2} m \left(\frac{du}{dt} \right)^2 + \frac{1}{2} k u^2. \quad (23)$$

Dengan mengalikan persamaan gerak dengan du/dt , diperoleh

$$\frac{dE}{dt} = -\mu \left(\frac{du}{dt} \right)^2 \leq 0. \quad (24)$$

Persamaan ini menunjukkan bahwa peredam selalu mengurangi energi sistem. Akibatnya, amplitudo getaran mengecil terhadap waktu.

Notebook meninjau keadaan *under-damped*, yaitu $\delta < \omega_0$. Akar persamaan karakteristiknya adalah

$$r_{1,2} = -\delta \pm i\omega, \quad \omega = \sqrt{\omega_0^2 - \delta^2}. \quad (25)$$

Bagian imajiner menghasilkan osilasi, sedangkan bagian real negatif $-\delta$ menghasilkan selubung amplitudo $e^{-\delta t}$.

5.2 Syarat Awal

$$u(0) = 1, \quad \frac{du}{dt}(0) = 0. \quad (26)$$

5.3 Solusi Analitik

Solusi keadaan *under-damped* pada notebook ditulis dalam bentuk

$$u(t) = e^{-\delta t} (2A \cos(\phi + \omega t)), \quad (27)$$

di mana syarat awal menentukan

$$\phi = \tan^{-1}\left(-\frac{\delta}{\omega}\right), \quad A = \frac{1}{2 \cos \phi}. \quad (28)$$

Faktor A memastikan $u(0) = 1$, sedangkan nilai ϕ memastikan $u'(0) = 0$. Pada implementasi digunakan $m = 1$, $\delta = 2$, dan $\omega_0 = 20$. Oleh karena itu, $\mu = 2\delta = 4$, $k = \omega_0^2 = 400$, dan domain waktu yang diamati adalah $t \in [0, 1]$.

Untuk detail matematis lebih lanjut mengenai osilator harmonik teredam, dapat dilihat pada [5].

5.4 Formulasi PINN pada Notebook

Berbeda dengan masalah Poisson dan Bratu, notebook osilator tidak menggunakan *hard constraint*. Aproksimasi solusi diambil langsung dari keluaran jaringan,

$$\hat{u}(t; \theta) = u_{nn}(t; \theta). \quad (29)$$

Pelatihan menggabungkan sejumlah kecil sampel solusi analitik dengan residual fisika. Untuk $m = 1$, residual pada titik kolokasi t_i adalah

$$\mathcal{R}(t_i) = \frac{d^2 \hat{u}}{dt^2}(t_i) + \mu \frac{d\hat{u}}{dt}(t_i) + k\hat{u}(t_i). \quad (30)$$

Komponen *physics loss* dan *data loss* masing-masing ditulis sebagai

$$\mathcal{L}_{physics} = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{R}(t_i)|^2, \quad (31)$$

$$\mathcal{L}_{data} = \frac{1}{N_d} \sum_{j=1}^{N_d} |\hat{u}(t_j; \theta) - u(t_j)|^2. \quad (32)$$

Fungsi objektif yang benar-benar digunakan oleh kode adalah

$$\mathcal{L} = \mathcal{L}_{data} + \lambda_{physics} \mathcal{L}_{physics}, \quad \lambda_{physics} = 10^{-4}. \quad (33)$$

Sampel data mencakup $t = 0$, sehingga nilai awal perpindahan ikut diarahkan oleh *data loss*. Namun, syarat $u'(0) = 0$ tidak dimasukkan sebagai komponen *loss* terpisah dan tidak dipaksakan secara aljabar. Informasi tersebut masuk secara tidak langsung karena data pelatihan dibuat dari solusi analitik yang telah memenuhi kedua syarat awal.

5.5 Penjelasan Kode Langkah demi Langkah

5.5.1 Membangkitkan solusi analitik

Fungsi `oscillator` menerapkan rumus solusi analitik. Perintah `assert d < w0` memastikan parameter berada pada keadaan *under-damped*. Dalam kode, `d` menyatakan δ dan `w0` menyatakan ω_0 .

```

1 def oscillator(d, w0, x):
2     assert d < w0
3     w = np.sqrt(w0**2 - d**2)
4     phi = np.arctan(-d/w)
5     A = 1/(2*np.cos(phi))
6     cos = torch.cos(phi + w*x)
7     exp = torch.exp(-d*x)
8     return exp * 2 * A * cos

```

Listing 10: Solusi analitik osilator harmonik teredam

Walaupun variabel bebas pada notebook dinamai `x`, secara fisika variabel tersebut merepresentasikan waktu t .

5.5.2 Membangun neural network

Model menggunakan tiga lapisan tersembunyi dengan 25 neuron pada setiap lapisan dan aktivasi tanh. Lapisan keluaran bersifat linear, sehingga arsitekturnya adalah $[1, 25, 25, 25, 1]$.

```
1 self.net = nn.Sequential(  
2     nn.Linear(1, n_hidden), nn.Tanh(),  
3     nn.Linear(n_hidden, n_hidden), nn.Tanh(),  
4     nn.Linear(n_hidden, n_hidden), nn.Tanh(),  
5     nn.Linear(n_hidden, 1)  
6 )  
7  
8 def forward(self, x):  
9     return self.net(x)  
10  
11 model = PINN(n_hidden=25).to(device)
```

Listing 11: Arsitektur jaringan pada notebook osilator

Bobot dan bias lapisan keluaran selanjutnya dikalikan 0.01. Seperti pada masalah Bratu, langkah ini membuat prediksi awal jaringan beramplitudo kecil.

5.5.3 Mendefinisikan data loss dan physics loss

Data loss mengukur kecocokan prediksi dengan sampel solusi analitik. *Physics loss* menghitung residual persamaan gerak pada titik kolokasi. Fungsi *derivative* yang digunakan sama dengan implementasi Bratu.

```
1 def data_loss(model, x_data, y_data):  
2     yh = model(x_data)  
3     return torch.mean((yh - y_data)**2)  
4  
5 def physics_loss(model, x, mu, k):  
6     u = model(x)  
7     u_x = derivative(u, x)  
8     u_xx = derivative(u_x, x)  
9     f = u_xx + mu * u_x + k * u  
10    return torch.mean(f**2)
```

Listing 12: Dua komponen loss pada osilator

5.5.4 Menyiapkan data dan titik kolokasi

Solusi analitik dibangkitkan pada 500 titik. Ekspresi `x[0:200:20]` mengambil 10 sampel dari bagian awal domain sebagai data berlabel. Sementara itu, 30 titik yang tersebar pada seluruh domain dipakai untuk mengevaluasi hukum fisika.

```
1 d = 2
2 w0 = 20
3 x = torch.linspace(0, 1, 500).view(-1, 1)
4 y = oscillator(d, w0, x).view(-1, 1)
5
6 x_data = x[0:200:20]
7 y_data = y[0:200:20]
8 x_physics = torch.linspace(0, 1, 30).view(-1, 1)
9 x_physics.requires_grad_(True)
```

Listing 13: Data berlabel dan titik kolokasi osilator

Pemisahan ini memperlihatkan fungsi dua jenis informasi: data berlabel mengarahkan nilai solusi pada sebagian kecil domain, sedangkan titik kolokasi mengarahkan bentuk solusi agar konsisten dengan persamaan diferensial pada seluruh domain.

5.5.5 Optimasi tahap pertama dengan Adam

Parameter fisika dihitung dari δ dan ω_0 . Selama 20000 *epoch*, Adam meminimalkan jumlah berbobot antara kedua komponen *loss*.

```
1 mu = 2 * d
2 k = w0**2
3 optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
4 lambda_physics = 1e-4
5
6 for epoch in range(20000):
7     optimizer.zero_grad()
8     loss = (data_loss(model, x_data, y_data)
9             + lambda_physics
10            * physics_loss(model, x_physics, mu, k))
11     loss.backward()
12     optimizer.step()
```

Listing 14: Pelatihan awal menggunakan Adam

Adam memberikan pembaruan parameter berbiaya relatif rendah dan digunakan untuk membawa model menuju daerah parameter yang baik sebelum optimasi orde kuasi-kedua.

5.5.6 Penyempurnaan dengan L-BFGS

Notebook kemudian menggunakan model yang sama untuk tahap L-BFGS. Dengan demikian, tahap ini melanjutkan hasil pelatihan Adam, bukan memulai model baru.

```
1 optimizer = torch.optim.LBFGS(  
2     model.parameters(), max_iter=10000, history_size=100,  
3     tolerance_grad=1e-6, tolerance_change=1e-9,  
4     line_search_fn="strong_wolfe"  
5 )  
6  
7 def closure():  
8     optimizer.zero_grad()  
9     loss = (data_loss(model, x_data, y_data)  
10          + lambda_physics  
11          * physics_loss(model, x_physics, mu, k))  
12     loss.backward()  
13     loss_log.append(loss.item())  
14     return loss  
15  
16 loss = optimizer.step(closure)
```

Listing 15: Penyempurnaan solusi menggunakan L-BFGS

5.5.7 Mengukur galat solusi

Setelah pelatihan, notebook menghitung MSE dan galat relatif L^2 . MSE mengukur rata-rata kuadrat galat titik demi titik, sedangkan galat relatif L^2 membandingkan norma galat total terhadap norma solusi analitik.

```
1 with torch.no_grad():  
2     u_pred = model(x)  
3  
4 u_analytic = oscillator(d, w0, x)  
5 l2_error = (torch.linalg.norm(u_pred - u_analytic)  
6            / torch.linalg.norm(u_analytic))  
7 MSE = torch.mean((u_pred - u_analytic)**2)
```

Listing 16: Metrik evaluasi osilator

Plot terakhir membandingkan solusi analitik, prediksi jaringan, data berlabel, dan lokasi evaluasi residual fisika. Plot *loss* dalam skala logaritmik digunakan untuk melihat laju konvergensi selama evaluasi optimizer.

References

- [1] MathWorks, *Physics-Informed Neural Networks*, dokumentasi daring.
- [2] M. Raissi, P. Perdikaris, dan G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, vol. 378, hal. 686–707, 2019. <https://doi.org/10.1016/j.jcp.2018.10.045>
- [3] I. E. Lagaris, A. Likas, dan D. I. Fotiadis, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Transactions on Neural Networks, vol. 9, no. 5, hal. 987–1000, 1998. <https://doi.org/10.1109/72.712178>
- [4] MathWorks, *Solve Partial Differential Equations with L-BFGS Method and Deep Learning*, MATLAB Documentation, 2024. Dokumentasi daring.
- [5] B. Umläuft, *The Harmonic Oscillator*, beltoforion.de, diakses 2024. https://beltoforion.de/en/harmonic_oscillator/